

Final Project Code

Leo Zhou

2026-07-03

Contents

Project overview	2
Load packages	2
Read local CSV data	2
Merge data	3
Data cleaning and feature engineering	4
Create helper functions	4
Create returns, volatility, lags, and target variable	5
Exploratory data analysis	6
Missing values	6
S&P 500 over time	7
Daily returns	8
Current volatility and future volatility	9
VIX versus future realized volatility	10
Modeling setup	11
Baseline model	12
Linear regression benchmark	12
OLS coefficient plot	13
Machine learning models with VIX	14
Random forest with VIX	14
Random forest variable importance with VIX	15
Neural network with VIX	15
Extension: machine learning models without VIX	16
OLS without VIX	16
Random forest without VIX	17
Random forest variable importance without VIX	18
Neural network without VIX	18
Model evaluation	19
Direct comparison of ML models with and without VIX	21
Actual versus predicted volatility	22

Project overview

This project asks:

Can regression and machine learning models improve short-term volatility forecasts compared to a simple persistence baseline?

The goal is not to predict whether the stock market goes up or down. Instead, the goal is to predict whether the market will be calm or turbulent over the next few trading days. This is useful because volatility is closely connected to risk management, portfolio decisions, and investor uncertainty.

A natural baseline is to assume that future volatility will look like recent volatility. This makes sense because volatility is persistent: calm periods often follow calm periods, and turbulent periods often follow turbulent periods. However, financial markets also contain forward-looking information. The VIX, for example, is often interpreted as a market-based measure of expected volatility. This project therefore asks whether models that use variables like the VIX and interest rates can improve on a simple recent-volatility benchmark.

An extension to this asks a more specific question:

Do the machine learning models still perform well if the VIX is removed as a predictor?

This matters because including VIX can make the model look stronger, but VIX is already a volatility-related measure. Removing it lets me test whether random forests and neural networks are learning useful patterns from past returns, realized volatility, and interest-rate variables, or whether their performance mainly depends on the VIX.

Load packages

```
# Install these packages once if they are not already installed:
# install.packages(c("tidyverse", "randomForest", "nnet", "knitr"))
# install.packages("gt")

library(tidyverse)
library(randomForest)
library(nnet)
library(knitr)
library(gt)
```

Read local CSV data

The data is already available as local CSV files. This section only reads the local files and assigns clear variable names. Each dataset has a date column called `observation_date` and one value column.

```
sp500 <- read.csv("../1_data/SP500.csv", na.strings = c(".", "", "NA"))
vix <- read.csv("../1_data/VIXCLS.csv", na.strings = c(".", "", "NA"))
dgs10 <- read.csv("../1_data/DGS10.csv", na.strings = c(".", "", "NA"))
fed_funds <- read.csv("../1_data/DFF.csv", na.strings = c(".", "", "NA"))

names(sp500) <- c("observation_date", "SP500")
names(vix) <- c("observation_date", "VIXCLS")
names(dgs10) <- c("observation_date", "DGS10")
names(fed_funds) <- c("observation_date", "DFF")
```

```

sp500$observation_date <- as.Date(sp500$observation_date)
vix$observation_date <- as.Date(vix$observation_date)
dgs10$observation_date <- as.Date(dgs10$observation_date)
fed_funds$observation_date <- as.Date(fed_funds$observation_date)

sp500$SP500 <- as.numeric(sp500$SP500)
vix$VIXCLS <- as.numeric(vix$VIXCLS)
dgs10$DGS10 <- as.numeric(dgs10$DGS10)
fed_funds$DFF <- as.numeric(fed_funds$DFF)

head(sp500)

```

```

##   observation_date   SP500
## 1      2021-06-25  4280.70
## 2      2021-06-28  4290.61
## 3      2021-06-29  4291.80
## 4      2021-06-30  4297.50
## 5      2021-07-01  4319.94
## 6      2021-07-02  4352.34

```

```
head(vix)
```

```

##   observation_date VIXCLS
## 1      2021-06-24   15.97
## 2      2021-06-25   15.62
## 3      2021-06-28   15.76
## 4      2021-06-29   16.02
## 5      2021-06-30   15.83
## 6      2021-07-01   15.48

```

```
head(dgs10)
```

```

##   observation_date DGS10
## 1      2021-06-24   1.49
## 2      2021-06-25   1.54
## 3      2021-06-28   1.49
## 4      2021-06-29   1.49
## 5      2021-06-30   1.45
## 6      2021-07-01   1.48

```

```
head(fed_funds)
```

```

##   observation_date DFF
## 1      2021-06-24  0.1
## 2      2021-06-25  0.1
## 3      2021-06-26  0.1
## 4      2021-06-27  0.1
## 5      2021-06-28  0.1
## 6      2021-06-29  0.1

```

Merge data

Each row in the merged dataset represents one S&P 500 trading day. I use the S&P 500 dataset as the master dataset and merge the other variables by date.

```
market <- merge(sp500, vix, by = "observation_date", all.x = TRUE)
market <- merge(market, dgs10, by = "observation_date", all.x = TRUE)
market <- merge(market, fed_funds, by = "observation_date", all.x = TRUE)
```

```
market <- market %>% arrange(observation_date)
```

```
head(market)
```

```
##   observation_date  SP500 VIXCLS DGS10  DFF
## 1      2021-06-25 4280.70  15.62  1.54 0.10
## 2      2021-06-28 4290.61  15.76  1.49 0.10
## 3      2021-06-29 4291.80  16.02  1.49 0.10
## 4      2021-06-30 4297.50  15.83  1.45 0.08
## 5      2021-07-01 4319.94  15.48  1.48 0.10
## 6      2021-07-02 4352.34  15.07  1.44 0.10
```

```
summary(market)
```

```
##   observation_date      SP500      VIXCLS      DGS10
##   Min.   :2021-06-25   Min.   :3577     Min.   :11.86   Min.   :1.190
##   1st Qu.:2022-09-26   1st Qu.:4299     1st Qu.:15.44   1st Qu.:3.390
##   Median :2023-12-26   Median :4764     Median :17.90   Median :4.060
##   Mean   :2023-12-25   Mean   :5148     Mean   :19.19   Mean   :3.644
##   3rd Qu.:2025-03-26   3rd Qu.:5971     3rd Qu.:21.74   3rd Qu.:4.300
##   Max.   :2026-06-25   Max.   :7610     Max.   :52.33   Max.   :4.980
##                                     NA's   :50      NA's   :20      NA's   :57
##
##           DFF
##   Min.   :0.060
##   1st Qu.:3.080
##   Median :4.330
##   Mean   :3.574
##   3rd Qu.:5.080
##   Max.   :5.330
##   NA's   :1
```

Data cleaning and feature engineering

Create helper functions

The first helper function computes rolling volatility using the previous `window` days. The second helper function computes actual future volatility over the next `horizon` days. This future volatility variable is the outcome I am trying to predict.

```
# computes past volatility
rolling_sd <- function(x, window) {
  out <- rep(NA_real_, length(x))
  for(i in seq_along(x)){
    if(i >= window){
      vals <- x[(i - window + 1):i]
      if(sum(!is.na(vals)) == window){
        out[i] <- sd(vals)
      }
    }
  }
  return(out)
}
```

```

}

# computes future volatility, which is the prediction target
future_sd <- function(x, horizon) {
  out <- rep(NA_real_, length(x))
  n <- length(x)
  for(i in seq_len(n)){
    if(i + horizon <= n){
      vals <- x[(i + 1):(i + horizon)]
      if(sum(!is.na(vals)) == horizon){
        out[i] <- sd(vals)
      }
    }
  }
  return(out)
}

```

Create returns, volatility, lags, and target variable

The main outcome is `target_vol_5`, which is the realized volatility over the next 5 trading days. I annualize volatility by multiplying daily standard deviation by `sqrt(252)`, because there are roughly 252 trading days in a year.

```

market <- market %>%
  mutate(
    sp500_return = log(SP500 / lag(SP500)),
    abs_return = abs(sp500_return),
    vol_5 = rolling_sd(sp500_return, 5) * sqrt(252),
    vol_20 = rolling_sd(sp500_return, 20) * sqrt(252),
    target_vol_5 = future_sd(sp500_return, 5) * sqrt(252),
    abs_return_lag1 = lag(abs_return, 1),
    abs_return_lag2 = lag(abs_return, 2),
    vix_lag1 = lag(VIXCLS, 1),
    vol_5_lag1 = lag(vol_5, 1),
    vol_20_lag1 = lag(vol_20, 1))

head(market)

```

##	observation_date	SP500	VIXCLS	DGS10	DFF	sp500_return	abs_return
## 1	2021-06-25	4280.70	15.62	1.54	0.10	NA	NA
## 2	2021-06-28	4290.61	15.76	1.49	0.10	0.0023123664	0.0023123664
## 3	2021-06-29	4291.80	16.02	1.49	0.10	0.0002773114	0.0002773114
## 4	2021-06-30	4297.50	15.83	1.45	0.08	0.0013272329	0.0013272329
## 5	2021-07-01	4319.94	15.48	1.48	0.10	0.0052080550	0.0052080550
## 6	2021-07-02	4352.34	15.07	1.44	0.10	0.0074721182	0.0074721182
##	vol_5	vol_20	target_vol_5	abs_return_lag1	abs_return_lag2	vix_lag1	
## 1	NA	NA	0.04699044	NA	NA	NA	
## 2	NA	NA	NA	NA	NA	15.62	
## 3	NA	NA	NA	0.0023123664	NA	15.76	
## 4	NA	NA	NA	0.0002773114	0.0023123664	16.02	
## 5	NA	NA	NA	0.0013272329	0.0002773114	15.83	
## 6	0.04699044	NA	NA	0.0052080550	0.0013272329	15.48	
##	vol_5_lag1	vol_20_lag1					
## 1	NA	NA					

```
## 2      NA      NA
## 3      NA      NA
## 4      NA      NA
## 5      NA      NA
## 6      NA      NA
```

```
summary(market)
```

```
## observation_date      SP500      VIXCLS      DGS10
## Min.   :2021-06-25  Min.   :3577  Min.   :11.86  Min.   :1.190
## 1st Qu.:2022-09-26  1st Qu.:4299  1st Qu.:15.44  1st Qu.:3.390
## Median :2023-12-26  Median :4764  Median :17.90  Median :4.060
## Mean   :2023-12-25  Mean   :5148  Mean   :19.19  Mean   :3.644
## 3rd Qu.:2025-03-26  3rd Qu.:5971  3rd Qu.:21.74  3rd Qu.:4.300
## Max.   :2026-06-25  Max.   :7610  Max.   :52.33  Max.   :4.980
##                NA's   :50    NA's   :20    NA's   :57
##      DFF      sp500_return      abs_return      vol_5
## Min.   :0.060  Min.   : -0.061609  Min.   :0.000003  Min.   :0.01277
## 1st Qu.:3.080  1st Qu.: -0.004699  1st Qu.:0.002492  1st Qu.:0.08830
## Median :4.330  Median : 0.000918  Median :0.005719  Median :0.13029
## Mean   :3.574  Mean   : 0.000556  Mean   :0.007659  Mean   :0.14960
## 3rd Qu.:5.080  3rd Qu.: 0.006358  3rd Qu.:0.010736  3rd Qu.:0.18638
## Max.   :5.330  Max.   : 0.090895  Max.   :0.090895  Max.   :0.95505
## NA's   :1      NA's   :101    NA's   :101    NA's   :300
##      vol_20      target_vol_5      abs_return_lag1      abs_return_lag2
## Min.   :0.0580  Min.   :0.01277  Min.   :0.000003  Min.   :0.000003
## 1st Qu.:0.1100  1st Qu.:0.08830  1st Qu.:0.002494  1st Qu.:0.002496
## Median :0.1406  Median :0.13029  Median :0.005726  Median :0.005730
## Mean   :0.1602  Mean   :0.14960  Mean   :0.007665  Mean   :0.007670
## 3rd Qu.:0.1936  3rd Qu.:0.18638  3rd Qu.:0.010737  3rd Qu.:0.010739
## Max.   :0.4833  Max.   :0.95505  Max.   :0.090895  Max.   :0.090895
## NA's   :844    NA's   :300    NA's   :102    NA's   :103
##      vix_lag1      vol_5_lag1      vol_20_lag1
## Min.   :11.86  Min.   :0.01277  Min.   :0.0580
## 1st Qu.:15.44  1st Qu.:0.08830  1st Qu.:0.1100
## Median :17.90  Median :0.13029  Median :0.1406
## Mean   :19.19  Mean   :0.14960  Mean   :0.1602
## 3rd Qu.:21.74  3rd Qu.:0.18638  3rd Qu.:0.1936
## Max.   :52.33  Max.   :0.95505  Max.   :0.4833
## NA's   :20    NA's   :300    NA's   :844
```

Exploratory data analysis

Missing values

```
colSums(is.na(market))
```

```
## observation_date      SP500      VIXCLS      DGS10
##              0              50              20              57
##      DFF      sp500_return      abs_return      vol_5
##              1              101              101              300
##      vol_20      target_vol_5      abs_return_lag1      abs_return_lag2
##              844              300              102              103
##      vix_lag1      vol_5_lag1      vol_20_lag1
```

##

20

300

844

The missing values in this dataset are expected and are not necessarily a data quality problem. This project uses daily financial time-series data, and the different series do not all follow the exact same calendar. For example, the S&P 500, VIX, Treasury yields, and federal funds rate can have missing values on holidays, weekends, or other non-trading days.

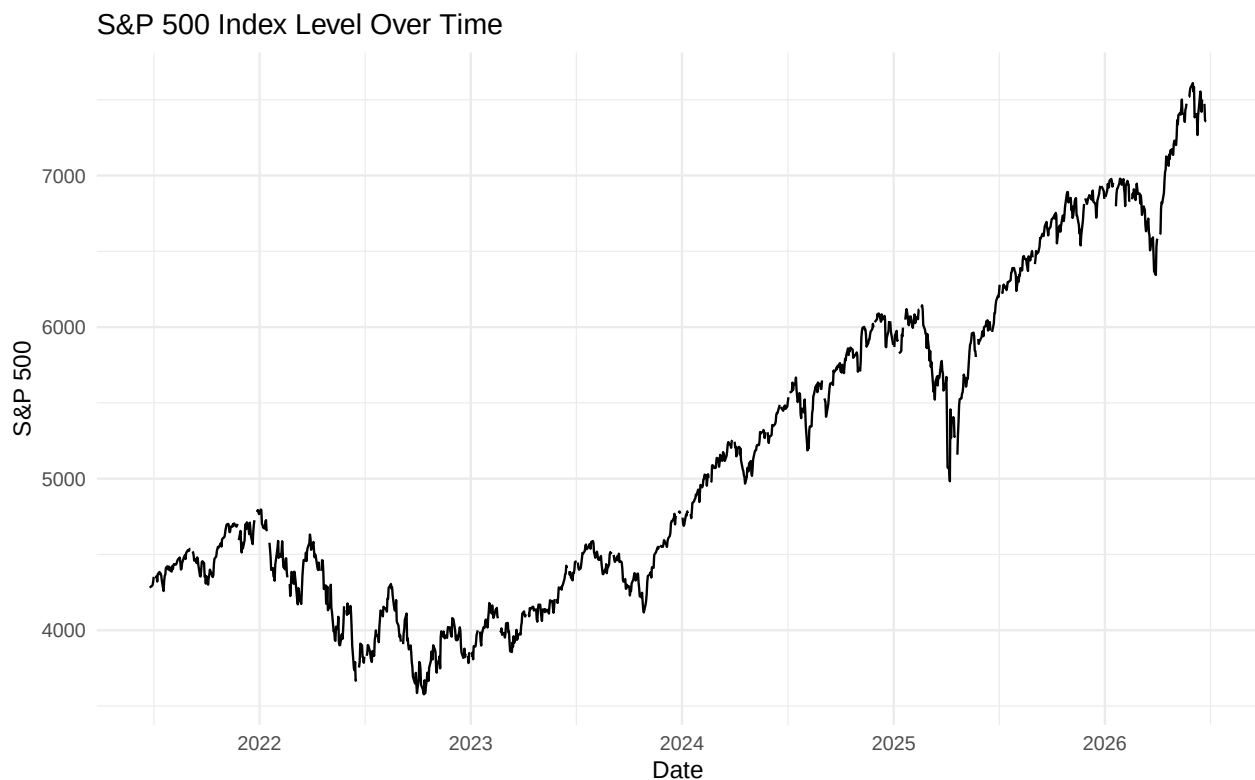
Some missing values are also created by variable construction. Daily returns require the previous day's price, so the first return is missing. Lagged variables require previous observations, so the first few rows cannot have lag values. Rolling volatility measures, such as 5-day and 20-day volatility, require a full window of past returns. Therefore, the first several observations cannot have rolling volatility values. The future volatility target also requires future returns, so the final few rows cannot have the target value.

The larger number of missing values for `vol_20` is especially expected because a 20-day rolling volatility measure needs 20 valid return observations in a row. If there is a missing return inside that 20-day window, the rolling volatility value for that window is also missing. This means a small number of missing price observations can create a larger number of missing rolling-volatility observations.

These missing values are acceptable because the models are not estimated using incomplete rows. Before running regressions or machine learning models, the project filters the data to complete observations for the variables used in the analysis. In other words, the missing values appear during the data construction stage, but they are removed before model estimation and prediction.

S&P 500 over time

```
ggplot(market, aes(x = observation_date, y = SP500)) +  
  geom_line() +  
  labs(title = "S&P 500 Index Level Over Time",  
        x = "Date", y = "S&P 500") +  
  theme_minimal()
```

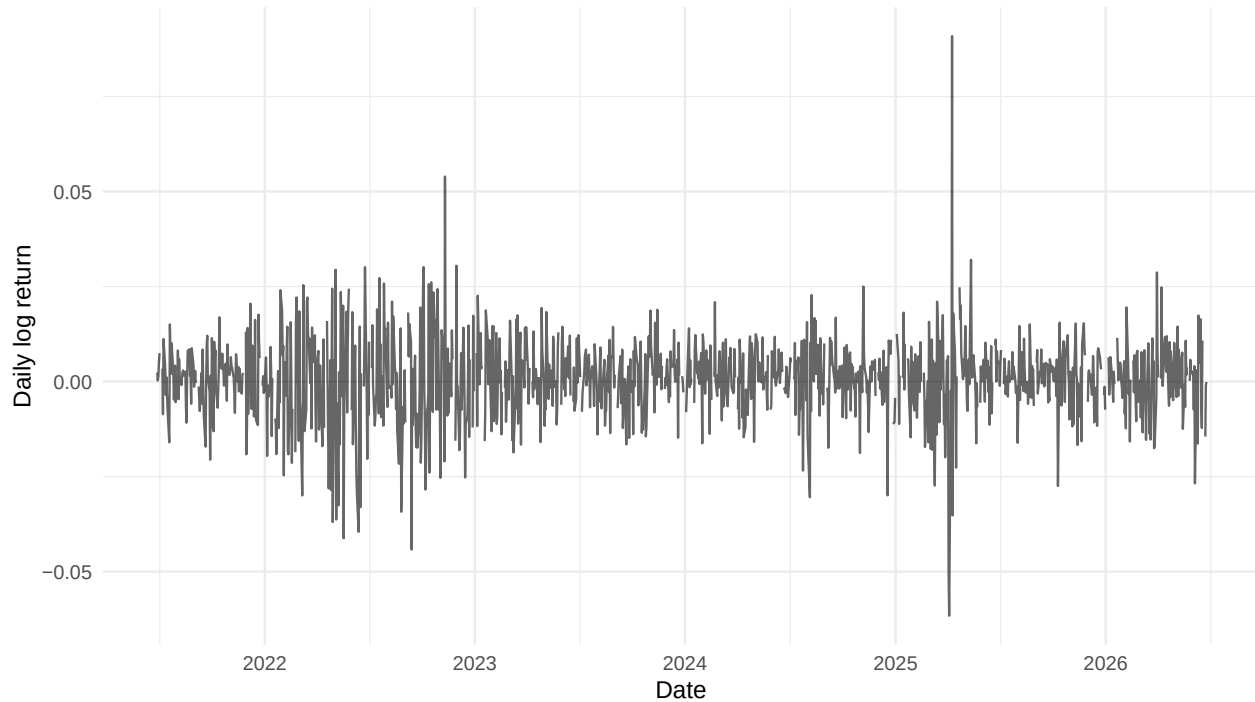


Daily returns

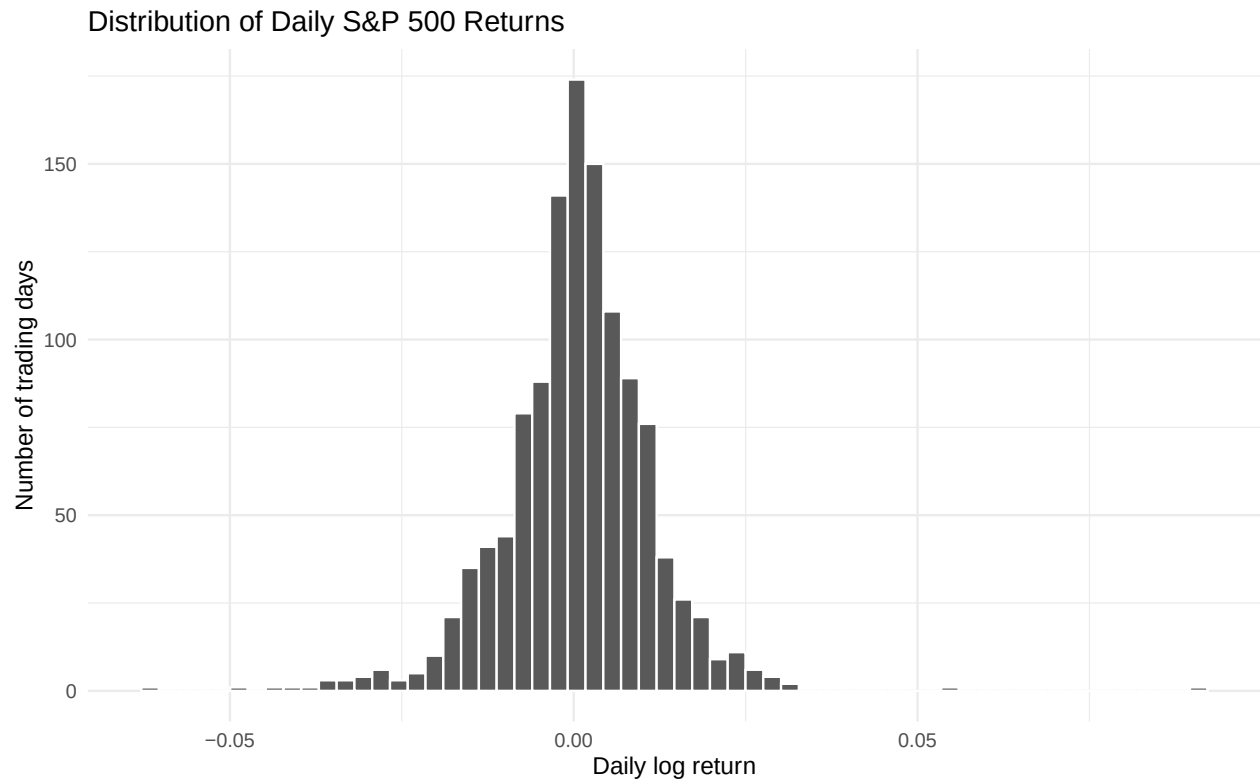
```
ggplot(market, aes(x = observation_date, y = sp500_return)) +  
  geom_line(alpha = 0.6) +  
  labs(title = "Daily S&P 500 Log Returns",  
        subtitle = "Large positive and negative spikes indicate turbulent periods.",  
        x = "Date", y = "Daily log return") +  
  theme_minimal()
```

Daily S&P 500 Log Returns

Large positive and negative spikes indicate turbulent periods.



```
ggplot(market, aes(x = sp500_return)) +  
  geom_histogram(bins = 60, color = "white") +  
  labs(title = "Distribution of Daily S&P 500 Returns",  
        x = "Daily log return", y = "Number of trading days") +  
  theme_minimal()
```

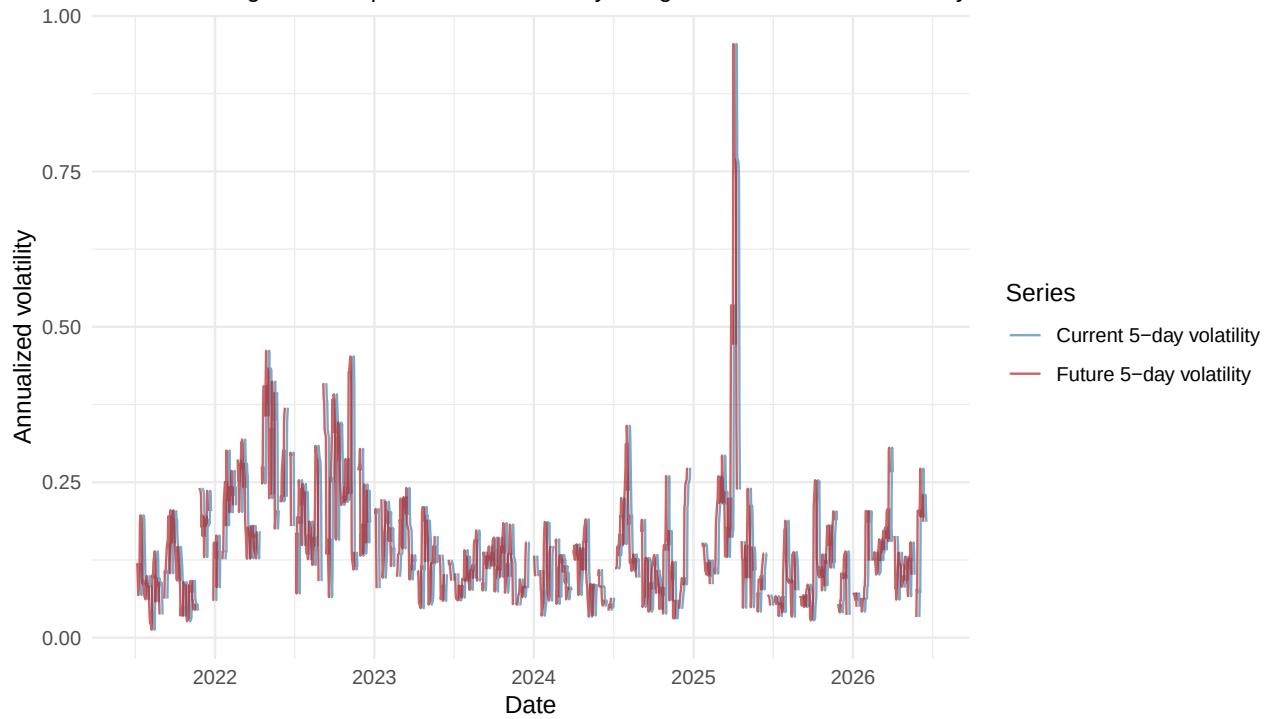



Current volatility and future volatility

```
ggplot(market, aes(x = observation_date)) +
  geom_line(aes(y = vol_5, color = "Current 5-day volatility"), alpha = 0.7) +
  geom_line(aes(y = target_vol_5, color = "Future 5-day volatility"), alpha = 0.7) +
  labs(title = "Current 5-Day Volatility and Future 5-Day Volatility",
       subtitle = "The forecasting task is to predict future volatility using information available today.",
       x = "Date", y = "Annualized volatility", color = "Series") +
  scale_color_manual(values = c(
    "Current 5-day volatility" = "steelblue", "Future 5-day volatility" = "firebrick")) +
  theme_minimal()
```

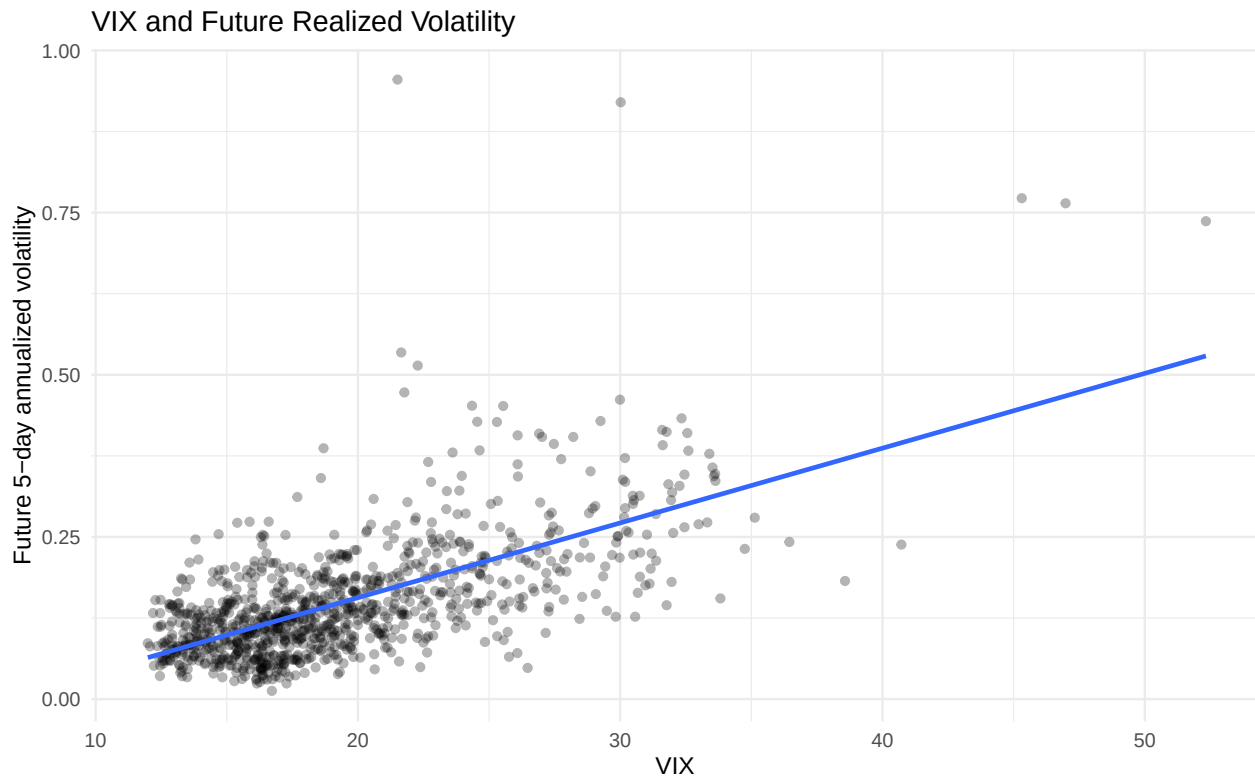
Current 5-Day Volatility and Future 5-Day Volatility

The forecasting task is to predict future volatility using information available today.



VIX versus future realized volatility

```
ggplot(market, aes(x = VIXCLS, y = target_vol_5)) +  
  geom_point(alpha = 0.3) +  
  geom_smooth(method = "lm", se = FALSE) +  
  labs(title = "VIX and Future Realized Volatility",  
       x = "VIX", y = "Future 5-day annualized volatility") +  
  theme_minimal()
```



Modeling setup

The models use only information available at time t to predict volatility after time t . I split the data chronologically: the first 80% is training data and the last 20% is testing data. This avoids randomly mixing past and future observations.

The first set of features includes VIX. The extension removes VIX and `vix_lag1` from the machine learning models to see whether the ML models are still useful without a direct volatility index.

```
features <- c(
  "vol_5",
  "vol_20",
  "abs_return",
  "abs_return_lag1",
  "abs_return_lag2",
  "VIXCLS",
  "vix_lag1",
  "DGS10",
  "DFF"
)

features_no_vix <- c(
  "vol_5",
  "vol_20",
  "abs_return",
  "abs_return_lag1",
  "abs_return_lag2",
  "DGS10",
  "DFF"
)
```

```

)

model_data <- market %>%
  select(observation_date, target_vol_5, all_of(features)) %>%
  drop_na()

split_index <- floor(0.80 * nrow(model_data))

train <- model_data[1:split_index, ]
test <- model_data[(split_index + 1):nrow(model_data), ]

nrow(train)

## [1] 270
nrow(test)

## [1] 68
range(train$observation_date)

## [1] "2021-08-03" "2025-04-10"
range(test$observation_date)

## [1] "2025-08-04" "2026-05-15"

```

Baseline model

The baseline model is a persistence model:

Future volatility is predicted to equal current 5-day volatility.

This baseline is simple but meaningful because volatility is persistent. If the market is turbulent now, it often remains turbulent in the near future.

```
test$pred_baseline <- test$vol_5
```

Linear regression benchmark

This is the econometric benchmark. It asks whether current volatility, VIX, Treasury rates, federal funds rates, and lagged absolute returns help predict future volatility.

```

ols_formula <- as.formula(paste("target_vol_5 ~", paste(features, collapse = " + ")))

ols_fit <- lm(ols_formula, data = train)
summary(ols_fit)

##
## Call:
## lm(formula = ols_formula, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.28906 -0.04206 -0.00782  0.02332  0.74815
##
## Coefficients:

```

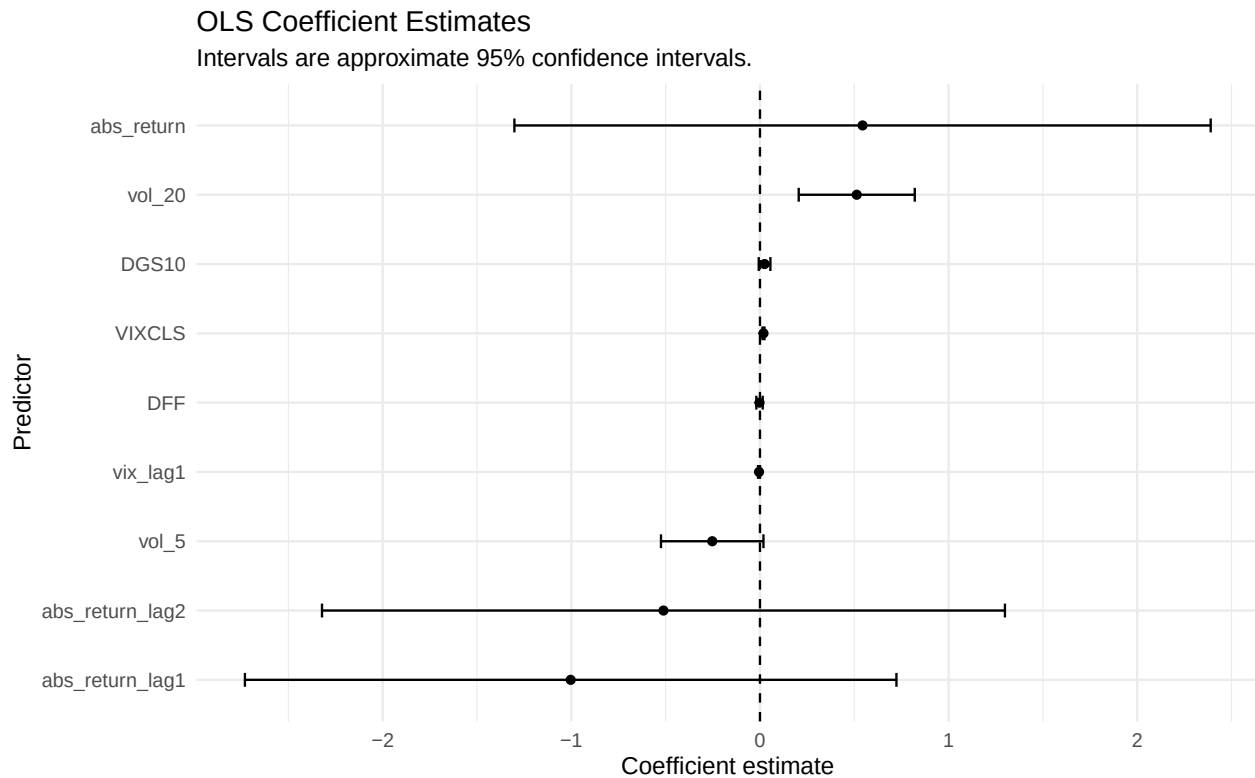
```
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.233627  0.034540 -6.764 8.84e-11 ***
## vol_5       -0.253075  0.138573 -1.826 0.06895 .
## vol_20      0.513034  0.156973  3.268 0.00123 **
## abs_return   0.543893  0.941759  0.578 0.56408
## abs_return_lag1 -1.003598  0.881211 -1.139 0.25580
## abs_return_lag2 -0.511620  0.923659 -0.554 0.58012
## VIXCLS      0.018650  0.002627  7.100 1.19e-11 ***
## vix_lag1    -0.004962  0.002782 -1.784 0.07566 .
## DGS10       0.024388  0.015792  1.544 0.12372
## DFF        -0.002264  0.008611 -0.263 0.79285
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.0933 on 260 degrees of freedom
## Multiple R-squared:  0.4956, Adjusted R-squared:  0.4781
## F-statistic: 28.38 on 9 and 260 DF,  p-value: < 2.2e-16
test$pred_ols <- predict(ols_fit, newdata = test)
```

OLS coefficient plot

```
ols_coef <- as.data.frame(summary(ols_fit)$coefficients)
ols_coef$term <- rownames(ols_coef)
names(ols_coef)[1:4] <- c("estimate", "std_error", "t_value", "p_value")

ols_coef <- ols_coef %>%
  filter(term != "(Intercept)") %>%
  mutate(lower = estimate - 1.96 * std_error, upper = estimate + 1.96 * std_error)

ggplot(ols_coef, aes(x = estimate, y = reorder(term, estimate))) +
  geom_point() +
  geom_errorbarh(aes(xmin = lower, xmax = upper), height = 0.2) +
  geom_vline(xintercept = 0, linetype = "dashed") +
  labs(title = "OLS Coefficient Estimates",
       subtitle = "Intervals are approximate 95% confidence intervals.",
       x = "Coefficient estimate", y = "Predictor") +
  theme_minimal()
```



Machine learning models with VIX

These models use the full feature set, including VIXCLS and vix_lag1.

Random forest with VIX

```
rf_fit <- randomForest::randomForest(
  ols_formula,
  data = train,
  ntree = 500,
  importance = TRUE
)

print(rf_fit)
```

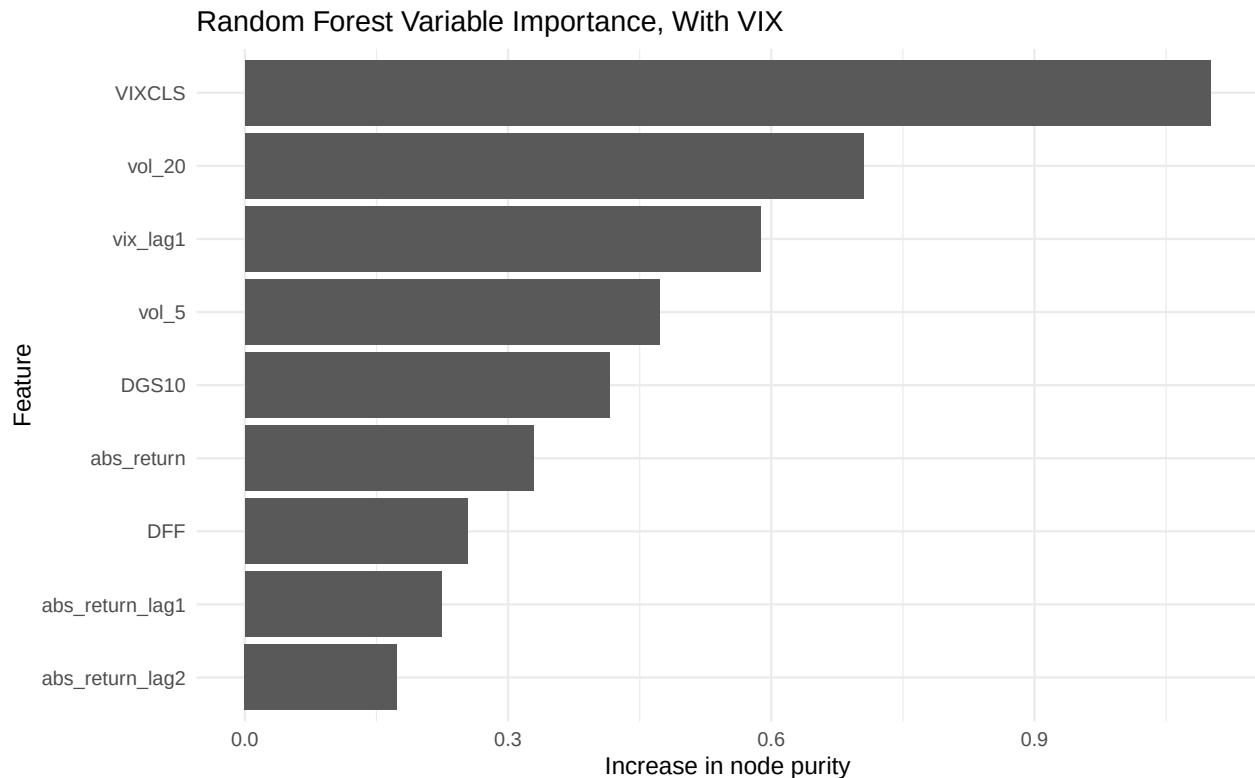
```
##
## Call:
## randomForest(formula = ols_formula, data = train, ntree = 500,      importance = TRUE)
##              Type of random forest: regression
##              Number of trees: 500
## No. of variables tried at each split: 3
##
##              Mean of squared residuals: 0.008395204
##              % Var explained: 49.48
```

```
test$pred_rf <- predict(rf_fit, newdata = test)
```

Random forest variable importance with VIX

```
importance_df <- as.data.frame(randomForest::importance(rf_fit))
importance_df$feature <- rownames(importance_df)

ggplot(importance_df, aes(x = reorder(feature, IncNodePurity), y = IncNodePurity)) +
  geom_col() +
  coord_flip() +
  labs(title = "Random Forest Variable Importance, With VIX",
       x = "Feature", y = "Increase in node purity") +
  theme_minimal()
```



Neural network with VIX

The neural network is more sensitive to scale than OLS or random forest, so I standardize the predictors and the outcome before fitting the model. The predictions are then converted back to the original volatility scale.

```
x_train <- train[, features]
x_test  <- test[, features]

x_scaled <- scale(x_train)
x_center <- attr(x_scaled, "scaled:center")
x_scale  <- attr(x_scaled, "scaled:scale")

x_train_scaled <- as.data.frame(x_scaled)
x_test_scaled  <- as.data.frame(scale(x_test, center = x_center, scale = x_scale))

y_center <- mean(train$target_vol_5)
y_scale  <- sd(train$target_vol_5)
```

```

y_train_scaled <- (train$target_vol_5 - y_center) / y_scale

nn_train <- cbind(target = y_train_scaled, x_train_scaled)

nn_fit <- nnet::nnet(target ~ ., data = nn_train, size = 5, decay = 0.01, linout = TRUE, maxit = 500, t

pred_nn_scaled <- predict(nn_fit, newdata = x_test_scaled)
test$pred_nn <- as.numeric(pred_nn_scaled) * y_scale + y_center

nn_with_vix_metrics <- tibble(
  model = "Neural network, with VIX",
  rmse = sqrt(mean((test$target_vol_5 - test$pred_nn)^2, na.rm = TRUE)),
  mae = mean(abs(test$target_vol_5 - test$pred_nn), na.rm = TRUE),
  mean_prediction = mean(test$pred_nn, na.rm = TRUE),
  mean_actual = mean(test$target_vol_5, na.rm = TRUE)
)

knitr::kable(nn_with_vix_metrics, digits = 4)

```

model	rmse	mae	mean_prediction	mean_actual
Neural network, with VIX	0.2859	0.1492	0.2525	0.1369

Extension: machine learning models without VIX

This extension uses the same training and testing rows as the models above, but removes `VIXCLS` and `vix_lag1` from the machine learning predictors. This makes the comparison cleaner because the only major change is whether the ML model gets access to the VIX variables.

If the no-VIX models perform similarly to the VIX models, that suggests the machine learning models are extracting useful information from recent returns, realized volatility, and interest-rate variables. If performance worsens substantially, that suggests VIX is doing a large share of the predictive work.

OLS without VIX

This model uses the same outcome and the same train/test split, but it removes both VIX variables from the predictors.

```

ols_formula_no_vix <- as.formula(
  paste("target_vol_5 ~", paste(features_no_vix, collapse = " + "))
)

ols_fit_no_vix <- lm(ols_formula_no_vix, data = train)
summary(ols_fit_no_vix)

```

```

##
## Call:
## lm(formula = ols_formula_no_vix, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.21286 -0.05187 -0.01503  0.02851  0.77832
##
## Coefficients:

```



```
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -0.073782   0.030259  -2.438   0.0154 *
## vol_5         -0.110866   0.146915  -0.755   0.4512
## vol_20         0.699012   0.176480   3.961 9.63e-05 ***
## abs_return     2.036958   1.049531   1.941   0.0534 .
## abs_return_lag1 0.689494   0.963829   0.715   0.4750
## abs_return_lag2 2.297478   0.991618   2.317   0.0213 *
## DGS10          0.038317   0.017873   2.144   0.0330 *
## DFF           -0.013635   0.009645  -1.414   0.1586
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1064 on 262 degrees of freedom
## Multiple R-squared:  0.3385, Adjusted R-squared:  0.3208
## F-statistic: 19.15 on 7 and 262 DF,  p-value: < 2.2e-16
```

```
test$pred_ols_no_vix <- predict(ols_fit_no_vix, newdata = test)

ols_no_vix_metrics <- tibble(
  model = "OLS regression, no VIX",
  rmse = sqrt(mean((test$target_vol_5 - test$pred_ols_no_vix)^2, na.rm = TRUE)),
  mae = mean(abs(test$target_vol_5 - test$pred_ols_no_vix), na.rm = TRUE),
  mean_prediction = mean(test$pred_ols_no_vix, na.rm = TRUE),
  mean_actual = mean(test$target_vol_5, na.rm = TRUE)
)

knitr::kable(ols_no_vix_metrics, digits = 4)
```

model	rmse	mae	mean_prediction	mean_actual
OLS regression, no VIX	0.0637	0.0458	0.1319	0.1369

Random forest without VIX

```
rf_formula_no_vix <- as.formula(
  paste("target_vol_5 ~", paste(features_no_vix, collapse = " + "))
)

rf_fit_no_vix <- randomForest::randomForest(
  rf_formula_no_vix,
  data = train,
  ntree = 500,
  importance = TRUE
)

print(rf_fit_no_vix)

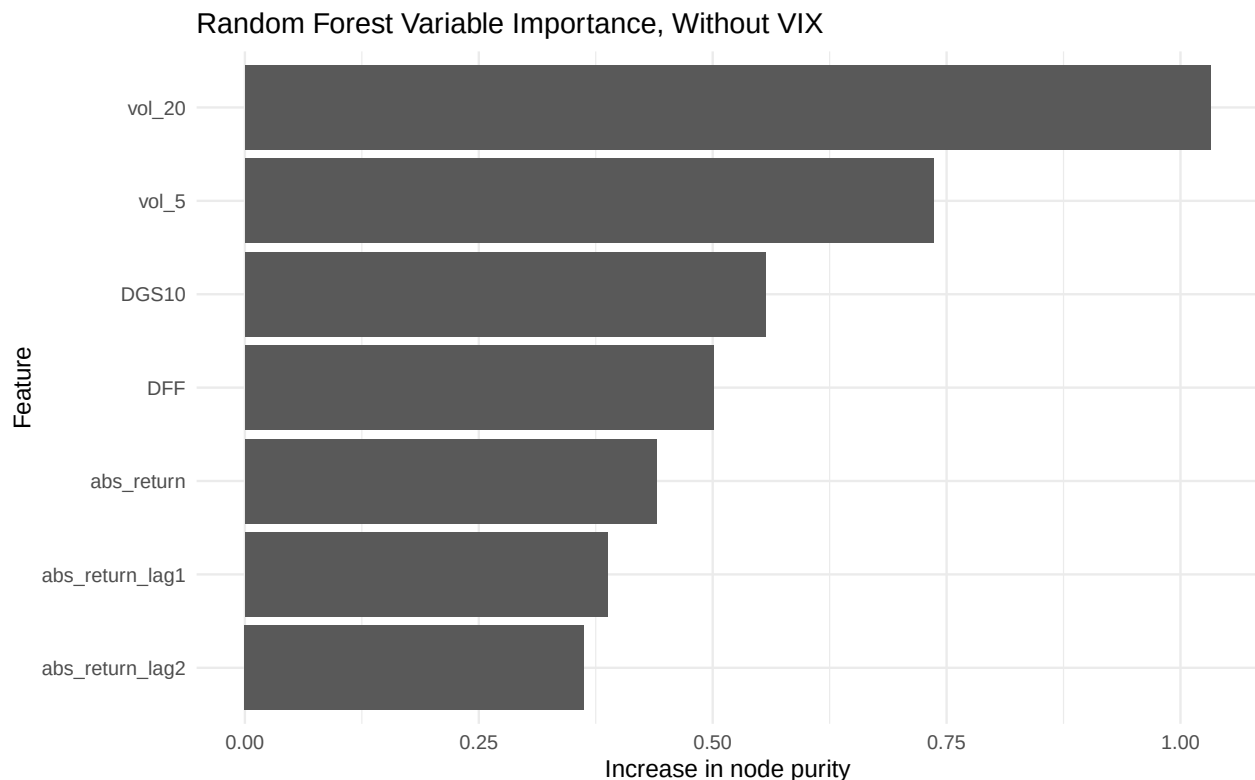
##
## Call:
## randomForest(formula = rf_formula_no_vix, data = train, ntree = 500,      importance = TRUE)
##               Type of random forest: regression
##               Number of trees: 500
## No. of variables tried at each split: 2
##
```

```
##           Mean of squared residuals: 0.009658122
##           % Var explained: 41.89
test$pred_rf_no_vix <- predict(rf_fit_no_vix, newdata = test)
```

Random forest variable importance without VIX

```
importance_no_vix_df <- as.data.frame(randomForest::importance(rf_fit_no_vix))
importance_no_vix_df$feature <- rownames(importance_no_vix_df)

ggplot(importance_no_vix_df, aes(x = reorder(feature, IncNodePurity), y = IncNodePurity)) +
  geom_col() +
  coord_flip() +
  labs(title = "Random Forest Variable Importance, Without VIX",
       x = "Feature", y = "Increase in node purity") +
  theme_minimal()
```



Neural network without VIX

```
x_train_no_vix <- train[, features_no_vix]
x_test_no_vix <- test[, features_no_vix]

x_no_vix_scaled <- scale(x_train_no_vix)
x_no_vix_center <- attr(x_no_vix_scaled, "scaled:center")
x_no_vix_scale <- attr(x_no_vix_scaled, "scaled:scale")

x_train_no_vix_scaled <- as.data.frame(x_no_vix_scaled)
x_test_no_vix_scaled <- as.data.frame(scale(
```

```

x_test_no_vix,
center = x_no_vix_center,
scale = x_no_vix_scale
))

nn_train_no_vix <- cbind(target = y_train_scaled, x_train_no_vix_scaled)

nn_fit_no_vix <- nnet::nnet(target ~ ., data = nn_train_no_vix, size = 5, decay = 0.01, linout = TRUE, n
pred_nn_no_vix_scaled <- predict(nn_fit_no_vix, newdata = x_test_no_vix_scaled)
test$pred_nn_no_vix <- as.numeric(pred_nn_no_vix_scaled) * y_scale + y_center

nn_no_vix_metrics <- tibble(
  model = "Neural network, no VIX",
  rmse = sqrt(mean((test$target_vol_5 - test$pred_nn_no_vix)^2, na.rm = TRUE)),
  mae = mean(abs(test$target_vol_5 - test$pred_nn_no_vix), na.rm = TRUE),
  mean_prediction = mean(test$pred_nn_no_vix, na.rm = TRUE),
  mean_actual = mean(test$target_vol_5, na.rm = TRUE)
)

knitr::kable(nn_no_vix_metrics, digits = 4)

```

model	rmse	mae	mean_prediction	mean_actual
Neural network, no VIX	0.1856	0.1478	0.2638	0.1369

Model evaluation

The main question is whether regression or machine learning beats the baseline out of sample. I compare models using RMSE and MAE. Lower values mean better predictive performance.

```

rmse <- function(actual, predicted) {
  sqrt(mean((actual - predicted)^2, na.rm = TRUE))
}

mae <- function(actual, predicted) {
  mean(abs(actual - predicted), na.rm = TRUE)
}

metrics <- tibble(
  model = c(
    "Baseline: current volatility",
    "OLS regression, with VIX",
    "OLS regression, no VIX",
    "Random forest, with VIX",
    "Random forest, no VIX",
    "Neural network, with VIX",
    "Neural network, no VIX"
  ),
  rmse = c(
    rmse(test$target_vol_5, test$pred_baseline),
    rmse(test$target_vol_5, test$pred_ols),
    rmse(test$target_vol_5, test$pred_ols_no_vix),

```

```

rmse(test$target_vol_5, test$pred_rf),
rmse(test$target_vol_5, test$pred_rf_no_vix),
rmse(test$target_vol_5, test$pred_nn),
rmse(test$target_vol_5, test$pred_nn_no_vix)
),
mae = c(
  mae(test$target_vol_5, test$pred_baseline),
  mae(test$target_vol_5, test$pred_ols),
  mae(test$target_vol_5, test$pred_ols_no_vix),
  mae(test$target_vol_5, test$pred_rf),
  mae(test$target_vol_5, test$pred_rf_no_vix),
  mae(test$target_vol_5, test$pred_nn),
  mae(test$target_vol_5, test$pred_nn_no_vix)
)
) %>%
  arrange(rmse)

knitr::kable(metrics, digits = 4)

```

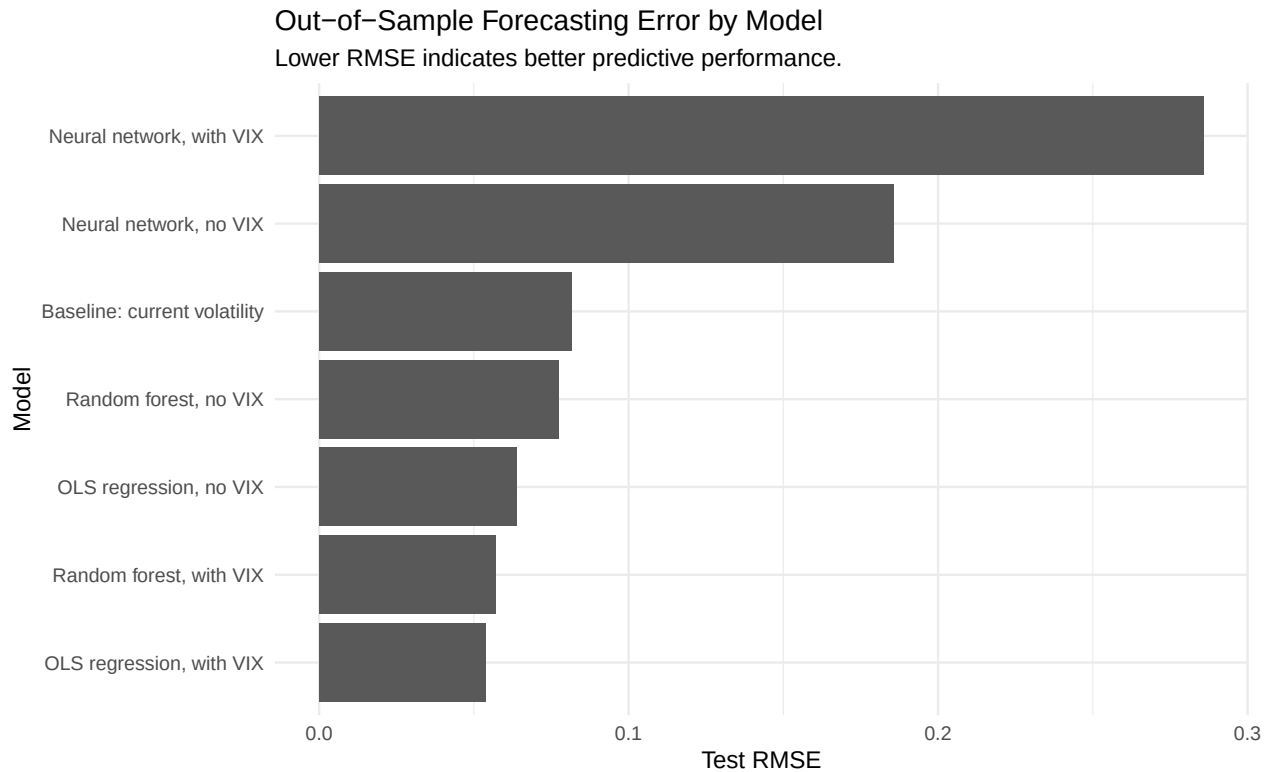
model	rmse	mae
OLS regression, with VIX	0.0537	0.0378
Random forest, with VIX	0.0569	0.0417
OLS regression, no VIX	0.0637	0.0458
Random forest, no VIX	0.0773	0.0655
Baseline: current volatility	0.0816	0.0619
Neural network, no VIX	0.1856	0.1478
Neural network, with VIX	0.2859	0.1492

```

# used LLMs to get code that saves the table result to a directory
table_path <- "../3_output/tables"
dir.create(table_path, recursive = TRUE, showWarnings = FALSE)
metrics %>% gt() %>%
  fmt_number(columns = where(is.numeric), decimals = 4) %>%
  gtsave(filename = file.path(table_path, "metrics.png"))

ggplot(metrics, aes(x = reorder(model, rmse), y = rmse)) +
  geom_col() +
  coord_flip() +
  labs(title = "Out-of-Sample Forecasting Error by Model",
       subtitle = "Lower RMSE indicates better predictive performance.",
       x = "Model", y = "Test RMSE") +
  theme_minimal()

```



Direct comparison of ML models with and without VIX

This table focuses only on whether removing the VIX changes the performance of the machine learning models.

```
ml_vix_comparison <- tibble(
  model = c("Random forest", "Neural network"),
  rmse_vix = c(
    rmse(test$target_vol_5, test$pred_rf),
    rmse(test$target_vol_5, test$pred_nn)
  ),
  rmse_no_vix = c(
    rmse(test$target_vol_5, test$pred_rf_no_vix),
    rmse(test$target_vol_5, test$pred_nn_no_vix)
  ),
  rmse_delta = rmse_no_vix - rmse_vix,
  mae_vix = c(
    mae(test$target_vol_5, test$pred_rf),
    mae(test$target_vol_5, test$pred_nn)
  ),
  mae_no_vix = c(
    mae(test$target_vol_5, test$pred_rf_no_vix),
    mae(test$target_vol_5, test$pred_nn_no_vix)
  ),
  mae_delta = mae_no_vix - mae_vix
)

knitr::kable(ml_vix_comparison, digits = 4)
```

model	rmse_vix	rmse_no_vix	rmse_delta	mae_vix	mae_no_vix	mae_delta
Random forest	0.0569	0.0773	0.0204	0.0417	0.0655	0.0238
Neural network	0.2859	0.1856	-0.1002	0.1492	0.1478	-0.0014

```
# used LLMs to get code that saves the table result to a directory
table_path <- "../3_output/tables"
dir.create(table_path, recursive = TRUE, showWarnings = FALSE)
ml_vix_comparison %>% gt() %>%
  fmt_number(columns = where(is.numeric), decimals = 4) %>%
  gtsave(filename = file.path(table_path, "ml_vix_comparison.png"))
```

Positive values in the change columns mean that removing VIX made the model worse. Negative values mean that removing VIX improved the model.

Actual versus predicted volatility

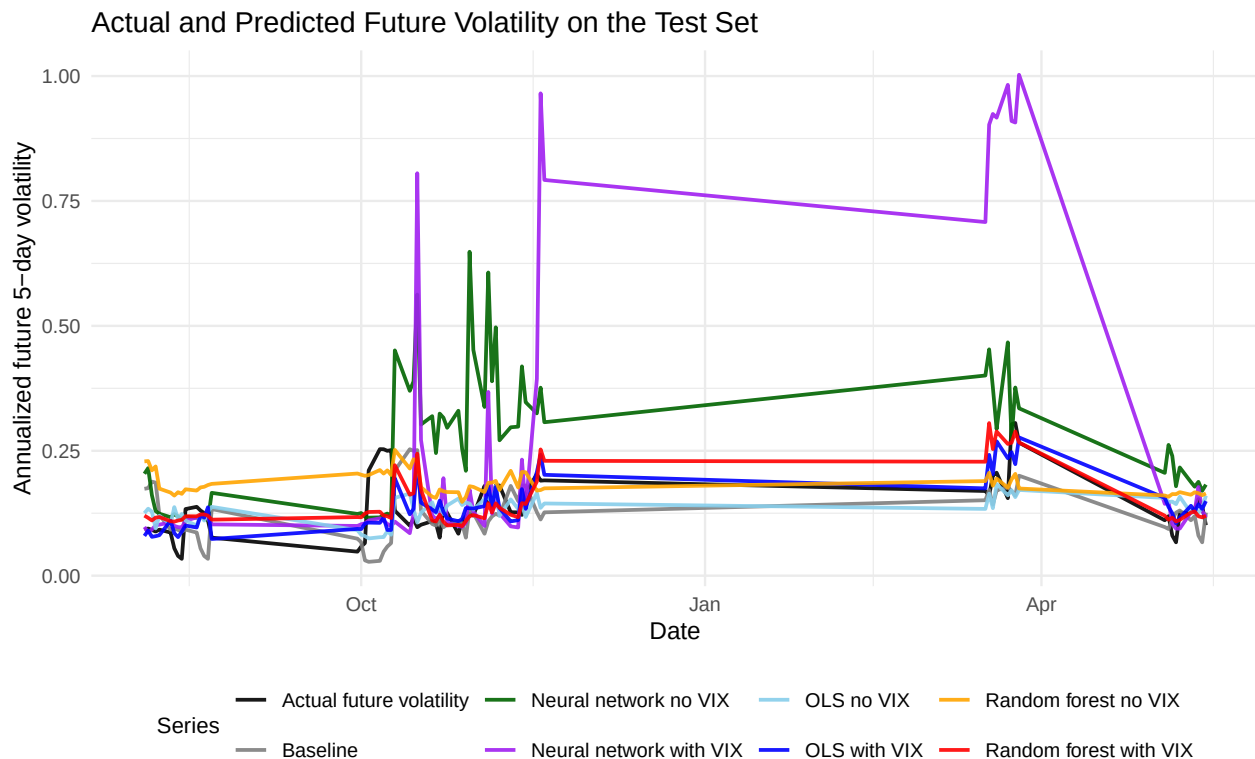
```
pred_long <- test %>%
  select(
    observation_date,
    actual = target_vol_5,
    baseline = pred_baseline,
    ols_with_vix = pred_ols,
    ols_no_vix = pred_ols_no_vix,
    random_forest_with_vix = pred_rf,
    random_forest_no_vix = pred_rf_no_vix,
    neural_network_with_vix = pred_nn,
    neural_network_no_vix = pred_nn_no_vix
  ) %>%
  pivot_longer(
    cols = -observation_date,
    names_to = "series",
    values_to = "volatility"
  ) %>%
  filter(!is.na(volatility))

ggplot(pred_long, aes(x = observation_date, y = volatility, color = series)) +
  geom_line(linewidth = 0.8, alpha = 0.9) +
  scale_color_manual(
    values = c(
      "actual" = "black",
      "baseline" = "gray50",
      "ols_with_vix" = "blue",
      "ols_no_vix" = "skyblue",
      "random_forest_with_vix" = "red",
      "random_forest_no_vix" = "orange",
      "neural_network_with_vix" = "purple",
      "neural_network_no_vix" = "darkgreen"
    ),
    labels = c(
      "actual" = "Actual future volatility",
      "baseline" = "Baseline",
      "ols_with_vix" = "OLS with VIX",
```

```

    "ols_no_vix" = "OLS no VIX",
    "random_forest_with_vix" = "Random forest with VIX",
    "random_forest_no_vix" = "Random forest no VIX",
    "neural_network_with_vix" = "Neural network with VIX",
    "neural_network_no_vix" = "Neural network no VIX"
  )
) +
labs(
  title = "Actual and Predicted Future Volatility on the Test Set",
  x = "Date", y = "Annualized future 5-day volatility",
  color = "Series") +
theme_minimal() +
theme(
  legend.position = "bottom",
  legend.title = element_text(size = 10),
  legend.text = element_text(size = 9)
)

```



```

pred_scatter <- test %>%
  select(
    actual = target_vol_5,
    baseline = pred_baseline,
    ols_with_vix = pred_ols,
    random_forest_with_vix = pred_rf,
    random_forest_no_vix = pred_rf_no_vix,
    neural_network_with_vix = pred_nn,
    neural_network_no_vix = pred_nn_no_vix
  ) %>%
  pivot_longer(
    cols = -actual,

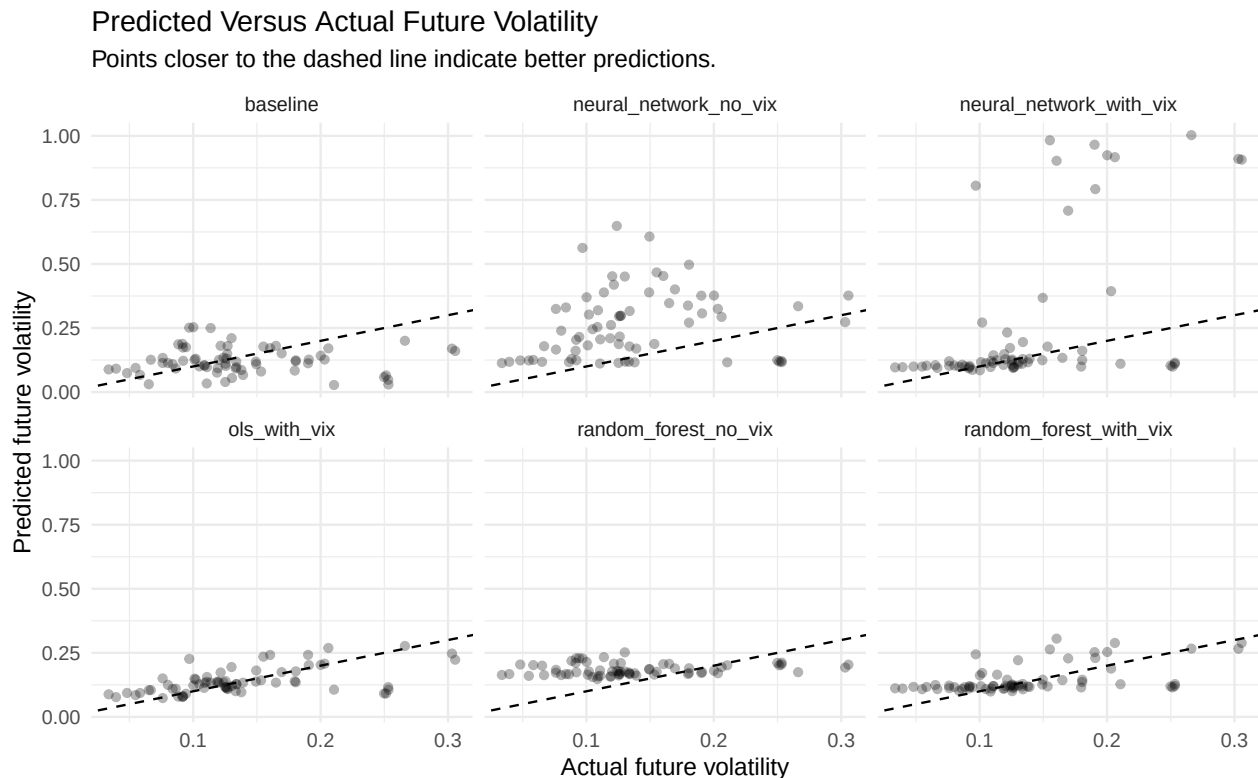
```

```

names_to = "model",
values_to = "prediction"
) %>%
filter(!is.na(prediction))

ggplot(pred_scatter, aes(x = actual, y = prediction)) +
  geom_point(alpha = 0.3) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed") +
  facet_wrap(~model) +
  labs(title = "Predicted Versus Actual Future Volatility",
       subtitle = "Points closer to the dashed line indicate better predictions.",
       x = "Actual future volatility", y = "Predicted future volatility") +
  theme_minimal()

```



Interpreting the results

The baseline model represents the idea that future volatility should look like recent realized volatility. This is a reasonable benchmark because volatility tends to persist over time. However, the baseline is not the best-performing model. Both the OLS model with VIX and the random forest model with VIX have lower RMSE and MAE than the baseline, which suggests that additional predictors contain useful forecasting information.

The strongest result is that the OLS regression with VIX performs best overall. It has the lowest RMSE and MAE, meaning its predictions are closest to actual future 5-day volatility in the test set. The random forest with VIX performs similarly but does not improve on the simpler OLS model.

The no-VIX comparison is important because it tests whether the models are mainly relying on the volatility index. For both OLS and random forest, removing VIX makes performance worse. This suggests that VIX adds meaningful predictive information beyond recent realized volatility, returns, and interest-rate variables.

In other words, the forward-looking information in VIX appears to help forecast future realized volatility.

At the same time, the machine learning models do not automatically outperform the simpler econometric model. The neural network performs much worse than the other models, likely because the dataset is noisy, the sample is limited, or the model is not tuned enough. This shows that added complexity can hurt forecasting performance when the model overfits or fails to learn a stable relationship.

Overall, the results suggest that VIX improves short-term volatility forecasts, but the best-performing model in this project is still a relatively simple OLS regression rather than a more complex machine learning model.